Revisión Técnica del Software — Fixpoint

- **Proyecto:** Fixpoint — Plataforma Web Centralizada para la Gestión de Soporte Técnico
- **Organización:** PyME de comercialización y mantenimiento de equipos TI
- **Código revisado:** módulo de ciclo de vida y asignación de tickets (models.py, forms.py, views.py, services.py, admin.py)
- **Fecha de revisión:** 03/11/2026
- **Modalidad:** revisión técnica de escritorio con listas de verificación, inspección de código, pruebas automatizadas y verificación empírica con reversión de datos
- **Alcance funcional:** RF-12, RF-21, RF-27, RF-28, RN-06, RN-07 y RN-08

Resumen ejecutivo

Se revisaron dos artefactos: el documento de especificación (requerimientos funcionales y reglas de negocio) y el componente que implementa el ciclo de vida del ticket. Se aplicaron **20 criterios** al documento y 20 criterios al código.

Checklist	Cumple	No cumple	No aplica	Total
Requerimientos	6	14	0	20
Código fuente	8	12	0	20

Se registraron **24 hallazgos**: 2 de severidad crítica, 5 mayores y 17 menores. Los dos hallazgos críticos son:

1. **HC-01:** la máquina de estados implementada permite cinco transiciones que la especificación prohíbe expresamente.
2. **HC-04:** el panel de administración de Django permite saltarse las reglas RN-06, RN-07, RN-08, RN-15 y RF-27 sin validación alguna.

Ningún hallazgo fue inferido sin evidencia: todos se respaldan con ubicación exacta (archivo:línea o documento:línea), inspección directa, prueba automatizada o reproducción controlada. La reproducción empírica se ejecutó dentro de una transacción revertida, por lo que no se modificó la base de datos de desarrollo.

1. Marco conceptual: la revisión técnica en el aseguramiento de la calidad

1.1 ¿Qué es una revisión técnica y cuál es su objetivo?

Una **revisión técnica** es un examen sistemático, documentado y colaborativo de un producto de trabajo de software —requerimientos, diseño, código, pruebas, documentación o configuración— realizado por personal calificado distinto de quien lo produjo. Su propósito es **identificar defectos, verificar la conformidad con la especificación y los estándares, y recomendar mejoras** antes de que el producto avance a la siguiente fase (IEEE 1028; ISO/IEC/IEEE 20246).

A diferencia de las pruebas, la revisión es una técnica estática: no requiere ejecutar el sistema y puede aplicarse a artefactos que aún no son ejecutables, como la especificación de requerimientos. Las pruebas observan el comportamiento; la revisión examina el artefacto y su correspondencia con lo esperado. Ambas técnicas son complementarias.

Objetivos principales:

- Detectar defectos de forma temprana, cuando corregirlos es más barato.
- Verificar que el artefacto cumple la especificación, la arquitectura y las convenciones acordadas.
- Evaluar la calidad del artefacto: claridad, completitud, consistencia, verificabilidad y mantenibilidad.
- Reducir el riesgo de retrabajo, incidentes en producción y defectos que escapan a las pruebas.
- Alinear al equipo y transferir conocimiento sobre el producto.
- Generar evidencia objetiva y métricas para la mejora continua del proceso.

1.2 Tipos de revisiones de software

Tipo	Quién la dirige	Formalidad	Propósito principal	Salida típica
Revisión informal	Un colega o autor	Baja	Detección rápida de errores evidentes	Comentarios
Walkthrough	El autor	Media	Explicar el artefacto y detectar problemas	Observaciones
Revisión técnica	Un moderador con pares	Media-alta	Evaluar la calidad técnica y la conformidad	Informe de revisión
Inspección formal	Moderador e inspectores	Alta	Detección sistemática con criterios y checklist	Registro formal

Tipo	Quién la dirige	Formalidad	Propósito principal	Salida típica
Revisión de gestión	Responsables del proyecto	Media	Evaluar avance, riesgos, costos y decisiones	Acta de decisiones
Auditoría	Auditor independiente	Alta	Verificar cumplimiento de estándares	Informe de auditoría

1.3 ¿Qué aspectos de un proyecto de software se pueden revisar?

Artefacto	Qué se revisa	Ejemplo en Fixpoint
Requerimientos	Claridad, completitud, consistencia	especificaci_n_del_proyecto.md
Diseño y arquitectura	Coherencia con requisitos, separación	Patrón MVT de Django
Código fuente	Nomenclatura, formato, responsabilidades	tickets/models.py
Modelo de datos	Entidades, relaciones, integridad	Ticket, Seguimiento
Pruebas	Cobertura, casos límite, aserciones	tickets/tests.py (40 pruebas)
Interfaz y UX	Claridad, accesibilidad, responsividad	Plantillas Bootstrap 5
Documentación	Exactitud, actualización, utilidad	README.md, DIAGRAMA_ER.md
Configuración	Versiones, reproducibilidad	requirements.txt, .env
Seguridad	Autenticación, autorización, CSRF	Login, RBAC, CsrfViewMiddleware
Rendimiento	Consultas, paginación	select_related, paginación
Despliegue	Instalación, migraciones, observabilidad	manage.py, migraciones

1.4 Beneficios de una revisión técnica

- Detección temprana de defectos.
- Reducción de costos por intervención temprana.
- Mayor calidad del producto, consistencia y trazabilidad.
- Mantenibilidad del código y reducción de riesgos técnicos.

1.5 Proceso y roles de una revisión técnica

1. Planificación
2. Apertura
3. Preparación individual
4. Reunión de revisión
5. Retrabajo
6. Seguimiento

Roles: moderador, autor, revisores y secretario.

2. Elementos seleccionados para la revisión

2.1 Nombre y propósito del proyecto

Fixpoint es una aplicación web desarrollada para una PyME de soporte técnico TI. Reemplaza la gestión informal de incidencias por una plataforma centralizada (Django 6.1.2, SQLite3, Bootstrap 5).

2.2 Principales funcionalidades del sistema

- Autenticación y gestión de roles.
- Creación y seguimiento de tickets por clientes.
- Tablero de tickets asignados, actualizaciones y cierres por agentes.
- Administración global de usuarios y reasignaciones por el Admin.
- Notificaciones internas, auditoría y SLAs de vencimiento.

2.3 Requerimientos que serán revisados

- **RF-12**: Al desactivar un agente, sus tickets abiertos regresan a estado Nuevo y sin asignar.
- **RF-21**: El Admin asigna o reasigna un ticket a un Agente.
- **RF-27**: Cambios de estado validados estrictamente.
- **RF-28**: El cierre exige registrar una resolución.
- **RN-06**: Solo agentes pueden ser asignados.
- **RN-07**: Ticket cerrado es solo lectura.
- **RN-08**: Resolución obligatoria de mínimo 10 caracteres al cerrar.

2.4 Módulo, clase o componente seleccionado

Componente de ciclo de vida y asignación de tickets:

- `tickets/models.py`: Modelo y máquina de estados.
- `tickets/forms.py`: Formularios de asignación y cierre.
- `tickets/views.py`: Vistas de detalle y asignación.
- `tickets/services.py`: Lógica de auditoría y liberación.

- `tickets/admin.py`: Panel de administración central.

4. Listas de verificación

4.1 Checklist para la revisión de requerimientos

Criterios de evaluación (Cumple/No cumple) sobre el documento original y las reglas definidas. Incluye: identificador único, claridad, falta de ambigüedad, verificabilidad, consistencia, factibilidad, roles y trazabilidad.

4.2 Checklist para la revisión del código fuente

Criterios de evaluación sobre el componente seleccionado. Incluye: PEP 8, nombres descriptivos, indentación, responsabilidad única, no duplicidad, validaciones, control de errores, seguridad del ORM, validación de dominio vs vistas y atomicidad.

5. Aplicación de la revisión técnica

5.1 Resultados — Checklist de requerimientos

Se reportaron faltas en: claridad y verificabilidad de términos, omisiones de requerimientos no funcionales (RNF), carencia de criterios de aceptación y problemas de formato documental.

5.2 Resultados — Checklist de código fuente

Se reportaron fallos críticos: las validaciones de negocio en el admin se pueden saltar, la máquina de estados en el modelo habilita transiciones erróneas (ej. Nuevo → Cerrado) y faltan transacciones atómicas para escrituras múltiples.

5.3 Registro de hallazgos (Resumen)

- **HC-01 (Crítica)**: Máquina de estados permite transiciones prohibidas por el negocio.
- **HC-04 (Crítica)**: Panel admin omite reglas del dominio.
- **HC-02 (Mayor)**: RF-12 incompleto, tickets "En Proceso" no regresan a "Nuevo".
- **HC-03 (Mayor)**: RN-08 no se valida directamente en el modelo de BD.
- **HR-01 y HR-02 (Mayor)**: Ausencia de requerimientos no funcionales y criterios de aceptación.

5.7 Conclusión y plan de acción

Se encontraron brechas críticas entre la especificación y la implementación. Acciones sugeridas:

1. Alinear tabla de estados del backend con la especificación de negocio.
2. Restringir y validar los guardados dentro del panel de administración (admin.py).
3. Asegurar validación en la capa del modelo de la longitud de resolución mínima.

4. Agregar transacciones atómicas y RNF medibles a la documentación.